

STEP 1 : Load The Data

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import matplotlib.ticker as ticker
```

Loading Both data into Pandas dataframe

```
customer_masterdata = pd.read_csv("C:\\Users\\Admin\\Desktop\\Python Project\\Dataset\\Customer_Master_Data.csv")
customer_masterdata
```

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus	NumChildren	JoinDate
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	0	2021-02-22
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Married	0	2023-12-06
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Widowed	2	2023-08-23
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divorced	2	2022-11-17
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divorced	0	2022-12-04
...	...	...	...	...	...	...	...	...	...
995	CUST10995	Mehul Chada	hridaanagate@hotmail.com	Male	70	Hyderabad	Divorced	2	2020-07-29
996	CUST10996	Arhaan Tara	qwali@mand-sood.com	Male	35	Delhi	Single	1	2022-07-24
997	CUST10997	Mahika Uppal	vdalal@yahoo.com	Female	70	Ahmedabad	Married	3	2023-01-27
998	CUST10998	Bhamini Aggarwal	kartik15@bajaj-singhal.com	Male	37	Jaipur	Single	0	2022-07-22
999	CUST10999	Alia Sekhon	urvichadha@hotmail.com	Male	67	Hyderabad	Widowed	1	2021-09-07

1000 rows × 9 columns

```
customer_transaction = pd.read_csv("C:\\Users\\Admin\\Desktop\\Python Project\\Dataset\\Customer_Transactions.csv")
customer_transaction
```

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	7/31/23	2383.07
1	CUST10100	3/10/24	497.54
2	CUST10031	2/17/25	536.78
3	CUST10987	7/17/23	314.89
4	CUST10831	12/15/24	2543.19
...	...	...	...
23045	CUST10710	3/11/24	931.09
23046	CUST10209	6/19/24	2659.35
23047	CUST10570	6/27/24	266.97
23048	CUST10075	12/26/23	1671.73
23049	CUST10234	7/7/24	981.32

23050 rows × 3 columns

Checking shape of the data frames

```
customer_masterdata.shape
```

```
(1000, 9)
```

```
customer_transaction.shape
```

```
(23050, 3)
```

Checking structure of the data frames

customer_masterdata.info

	DataFrame.info of		CustomerID	Name	Email	Gender	Age	\
0	CUST10000	Onkar Bhargava		pkeer@yahoo.com	Male	54		
1	CUST10001	Divit Kohli		mkalita@sarin.com	Female	48		
2	CUST10002	Kiara Behl		apteanay@hotmail.com	Male	75		
3	CUST10003	Vaibhav Sankar		bseshadri@choudhry.info	Male	62		
4	CUST10004	Shray D'Alia		bdhillon@toor-mall.com	Male	55		
..	...	...		...	...	...		
995	CUST10995	Mehul Chada		hridaanagate@hotmail.com	Male	70		
996	CUST10996	Arhaan Tara		qwali@mand-sood.com	Male	35		
997	CUST10997	Mahika Uppal		vdalal@yahoo.com	Female	70		
998	CUST10998	Bhamini Aggarwal		kartik15@bajaj-singhal.com	Male	37		
999	CUST10999	Alia Sekhon		urvichadha@hotmail.com	Male	67		

	City	MaritalStatus	NumChildren	JoinDate
0	Delhi	Divorced	0	2021-02-22
1	Kolkata	Married	0	2023-12-06
2	Kolkata	Widowed	2	2023-08-23
3	Pune	Divorced	2	2022-11-17
4	Delhi	Divorced	0	2022-12-04
..	...	...	...	...
995	Hyderabad	Divorced	2	2020-07-29
996	Delhi	Single	1	2022-07-24
997	Ahmedabad	Married	3	2023-01-27
998	Jaipur	Single	0	2022-07-22
999	Hyderabad	Widowed	1	2021-09-07

[1000 rows x 9 columns]>

```
customer_transaction.info
```

```
<bound method DataFrame.info of
0      CUST10771      7/31/23      2383.07
1      CUST10100      3/10/24      497.54
2      CUST10031      2/17/25      536.78
3      CUST10987      7/17/23      314.89
4      CUST10831      12/15/24      2543.19
...      ...      ...      ...
23045 CUST10710      3/11/24      931.09
23046 CUST10209      6/19/24      2659.35
23047 CUST10570      6/27/24      266.97
23048 CUST10075      12/26/23      1671.73
23049 CUST10234      7/7/24      981.32

[23050 rows x 3 columns]>
```

```
customer_masterdata.dtypes
```

```
CustomerID      str
Name            str
Email           str
Gender          str
Age             int64
City            str
MaritalStatus   str
NumChildren     int64
JoinDate        str
dtype: object
```

```
customer_transaction.dtypes
```

```
CustomerID      str
TransactionDate  str
TransactionAmount float64
dtype: object
```

Checking preview of the data frames

```
customer_masterdata.head()
```

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus	NumChildren	JoinDate
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	0	2021-02-22
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Married	0	2023-12-06
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Widowed	2	2023-08-23
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divorced	2	2022-11-17
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divorced	0	2022-12-04

```
customer_masterdata.tail()
```

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus	NumChildren	JoinDate
995	CUST10995	Mehul Chada	hridaanagate@hotmail.com	Male	70	Hyderabad	Divorced	2	2020-07-29
996	CUST10996	Arhaan Tara	qwali@mand-sood.com	Male	35	Delhi	Single	1	2022-07-24
997	CUST10997	Mahika Uppal	vdalal@yahoo.com	Female	70	Ahmedabad	Married	3	2023-01-27
998	CUST10998	Bhamini Aggarwal	kartik15@bajaj-singhal.com	Male	37	Jaipur	Single	0	2022-07-22
999	CUST10999	Alia Sekhon	urvichadha@hotmail.com	Male	67	Hyderabad	Widowed	1	2021-09-07

```
customer_transaction.head()
```

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	7/31/23	2383.07
1	CUST10100	3/10/24	497.54
2	CUST10031	2/17/25	536.78
3	CUST10987	7/17/23	314.89
4	CUST10831	12/15/24	2543.19

```
customer_transaction.tail()
```

	CustomerID	TransactionDate	TransactionAmount
23045	CUST10710	3/11/24	931.09
23046	CUST10209	6/19/24	2659.35
23047	CUST10570	6/27/24	266.97
23048	CUST10075	12/26/23	1671.73
23049	CUST10234	7/7/24	981.32

STEP 2: Clean The Data

Convert JoinDate and TransactionDate columns to datetime

```
customer_masterdata.columns
```

```
Index(['CustomerID', 'Name', 'Email', 'Gender', 'Age', 'City', 'MaritalStatus',  
      'NumChildren', 'JoinDate'],  
      dtype='str')
```

```
customer_transaction.columns
```

```
Index(['CustomerID', 'TransactionDate', 'TransactionAmount'], dtype='str')
```

```
customer_masterdata.dtypes
```

```
CustomerID      str  
Name            str  
Email           str  
Gender          str  
Age            int64  
City           str  
MaritalStatus   str  
NumChildren     int64  
JoinDate       str  
dtype: object
```

```
customer_masterdata['JoinDate'] = pd.to_datetime(customer_masterdata['JoinDate'])  
customer_masterdata.dtypes
```

```
CustomerID      str  
Name            str  
Email           str  
Gender          str  
Age            int64  
City           str  
MaritalStatus   str  
NumChildren     int64  
JoinDate       datetime64[us]  
dtype: object
```

customer_masterdata

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus	NumChildren	JoinDate
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	0	2021-02-22
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Married	0	2023-12-06
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Widowed	2	2023-08-23
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divorced	2	2022-11-17
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divorced	0	2022-12-04
...	...	...	...	...	...	...	...	...	...
995	CUST10995	Mehul Chada	hridaanagate@hotmail.com	Male	70	Hyderabad	Divorced	2	2020-07-29
996	CUST10996	Arhaan Tara	qwali@mand-sood.com	Male	35	Delhi	Single	1	2022-07-24
997	CUST10997	Mahika Uppal	vdalal@yahoo.com	Female	70	Ahmedabad	Married	3	2023-01-27
998	CUST10998	Bhamini Aggarwal	kartik15@bajaj-singhal.com	Male	37	Jaipur	Single	0	2022-07-22
999	CUST10999	Alia Sekhon	urvichadha@hotmail.com	Male	67	Hyderabad	Widowed	1	2021-09-07

1000 rows × 9 columns

customer_transaction.dtypes

```
CustomerID          str
TransactionDate      str
TransactionAmount    float64
dtype: object
```

```
customer_transaction['TransactionDate'] = pd.to_datetime(customer_transaction['TransactionDate'], format="%m/%d/%y")
customer_transaction.dtypes
```

```
CustomerID          str
TransactionDate      datetime64[us]
TransactionAmount    float64
dtype: object
```

customer_transaction

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	2023-07-31	2383.07
1	CUST10100	2024-03-10	497.54
2	CUST10031	2025-02-17	536.78
3	CUST10987	2023-07-17	314.89
4	CUST10831	2024-12-15	2543.19
...	...	...	...
23045	CUST10710	2024-03-11	931.09
23046	CUST10209	2024-06-19	2659.35
23047	CUST10570	2024-06-27	266.97
23048	CUST10075	2023-12-26	1671.73
23049	CUST10234	2024-07-07	981.32

23050 rows × 3 columns

Ensure no nulls or bad types

```
customer_masterdata.isnull().sum()
```

```
CustomerID      0  
Name            0  
Email          0  
Gender          0  
Age            0  
City           0  
MaritalStatus  0  
NumChildren    0  
JoinDate       0  
dtype: int64
```

```
customer_masterdata.isna().sum()
```

```
CustomerID      0  
Name            0  
Email          0  
Gender          0  
Age            0  
City           0  
MaritalStatus  0  
NumChildren    0  
JoinDate       0  
dtype: int64
```

```
customer_transaction.isnull().sum()
```

```
CustomerID      0  
TransactionDate 0  
TransactionAmount 0  
dtype: int64
```

```
customer_transaction.isna().sum()
```

```
CustomerID      0  
TransactionDate  0  
TransactionAmount  0  
dtype: int64
```

Validate uniqueness of CustomerID in master dataset

```
customer_masterdata['CustomerID'].duplicated().sum()
```

```
np.int64(0)
```

```
customer_transaction['CustomerID'].duplicated().sum()
```

```
np.int64(22050)
```

```
customer_transaction.duplicated().sum()
```

```
np.int64(0)
```

Ensure all transaction CustomerIDs exist in master data

```
customer_transaction['CustomerID'].count()
```

```
np.int64(23050)
```

```
customer_transaction['CustomerID'].isin(customer_masterdata['CustomerID']).sum()
```

```
np.int64(23050)
```

STEP 3: Merge If Needed

```
consolidated_data = pd.merge(customer_transaction, customer_masterdata)  
consolidated_data
```

	CustomerID	TransactionDate	TransactionAmount	Name	Email	Gender	Age	City	MaritalStatus	NumChildren	JoinDate
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon	dharmajantara@gmail.com	Female	27	Ahmedabad	Widowed	3	2023-05-31
1	CUST10100	2024-03-10	497.54	Aniruddh Borah	jivikabhavsar@gmail.com	Female	53	Lucknow	Divorced	4	2022-01-08
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja	jhaverifarhan@chandra.org	Male	40	Ahmedabad	Single	0	2023-12-03
3	CUST10987	2023-07-17	314.89	Jayan Wagle	ojas82@gmail.com	Not Disclosed	75	Bangalore	Widowed	4	2022-03-18
4	CUST10831	2024-12-15	2543.19	Ishita Agarwal	vbalay@yahoo.com	Not Disclosed	27	Jaipur	Divorced	4	2021-06-30
...	...	...	...	...	...	...	...	...	...	...	...
23045	CUST10710	2024-03-11	931.09	Divit Toor	saksham79@ram.biz	Not Disclosed	73	Mumbai	Widowed	1	2024-02-18
23046	CUST10209	2024-06-19	2659.35	Jivin Cheema	darivana@hotmail.com	Male	56	Pune	Married	1	2024-05-09
23047	CUST10570	2024-06-27	266.97	Advika Dyal	granganathan@sathe-dhar.org	Not Disclosed	26	Pune	Single	1	2021-03-29
23048	CUST10075	2023-12-26	1671.73	Amira Lall	vanyabiswas@kannan.biz	Female	29	Jaipur	Married	1	2023-02-25
23049	CUST10234	2024-07-07	981.32	Ishaan Ravel	bhardwajadah@yahoo.com	Not Disclosed	24	Jaipur	Widowed	2	2020-08-16

23050 rows × 11 columns

```
consolidated_data.shape
```

```
(23050, 11)
```

STEP 4: Perform RFM Calculations

```
referenceDate = consolidated_data['TransactionDate'].max() + pd.Timedelta(days=1)
referenceDate
```

```
Timestamp('2025-07-30 00:00:00')
```

```
consolidated_data.groupby(['CustomerID'])['TransactionDate'].max()
```

```
CustomerID
CUST10000    2025-07-17
CUST10001    2025-06-25
CUST10002    2025-07-12
CUST10003    2025-05-10
CUST10004    2025-07-22
...
CUST10995    2024-06-23
CUST10996    2025-07-15
CUST10997    2025-06-28
CUST10998    2025-03-26
CUST10999    2025-07-25
Name: TransactionDate, Length: 1000, dtype: datetime64[us]
```

```
consolidated_data.groupby(['CustomerID'])['TransactionDate'].count()
```

```
CustomerID
CUST10000    23
CUST10001    30
CUST10002    24
CUST10003    25
CUST10004    19
..
CUST10995    21
CUST10996    21
CUST10997    20
CUST10998    25
CUST10999    23
Name: TransactionDate, Length: 1000, dtype: int64
```

```
consolidated_data.groupby(['CustomerID'])['TransactionAmount'].sum()
```

```
CustomerID
CUST10000    21265.49
CUST10001    28654.31
CUST10002    23884.03
CUST10003    24206.03
CUST10004    25565.30
...
CUST10995    24325.19
CUST10996    21809.11
CUST10997    21120.48
CUST10998    29494.56
CUST10999    22028.01
Name: TransactionAmount, Length: 1000, dtype: float64
```

```
df_rfm = pd.DataFrame()
```

```
df_rfm = consolidated_data.groupby(['CustomerID']).agg(LastTransactionDay=('TransactionDate', 'max'), CountOfTransactions=
('TransactionDate', 'count'), SumOfTransactionAmount=('TransactionAmount', 'sum')).reset_index()
df_rfm
```

	CustomerID	LastTransactionDay	CountOfTransactions	SumOfTransactionAmount
0	CUST10000	2025-07-17	23	21265.49
1	CUST10001	2025-06-25	30	28654.31
2	CUST10002	2025-07-12	24	23884.03
3	CUST10003	2025-05-10	25	24206.03
4	CUST10004	2025-07-22	19	25565.30
...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19
996	CUST10996	2025-07-15	21	21809.11
997	CUST10997	2025-06-28	20	21120.48
998	CUST10998	2025-03-26	25	29494.56
999	CUST10999	2025-07-25	23	22028.01

1000 rows × 4 columns

Use a reference date to compute Recency in number of days

```
df_rfm['Recency'] = referenceDate-df_rfm['LastTransactionDay']
df_rfm
```

	CustomerID	LastTransactionDay	CountOfTransactions	SumOfTransactionAmount	Recency
0	CUST10000	2025-07-17	23	21265.49	13 days
1	CUST10001	2025-06-25	30	28654.31	35 days
2	CUST10002	2025-07-12	24	23884.03	18 days
3	CUST10003	2025-05-10	25	24206.03	81 days
4	CUST10004	2025-07-22	19	25565.30	8 days
...	...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19	402 days
996	CUST10996	2025-07-15	21	21809.11	15 days
997	CUST10997	2025-06-28	20	21120.48	32 days
998	CUST10998	2025-03-26	25	29494.56	126 days
999	CUST10999	2025-07-25	23	22028.01	5 days

1000 rows × 5 columns

Store result in a new DataFrame called df_rfm

```
df_rfm = df_rfm.rename(columns={'LastTransactionDay': 'Recency',
                                'CountOfTransactions': 'Frequency',
                                'SumOfTransactionAmount': 'Monetary',
                                'Recency': 'RecencyInDays'})

df_rfm
```

	CustomerID	Recency	Frequency	Monetary	RecencyInDays
0	CUST10000	2025-07-17	23	21265.49	13 days
1	CUST10001	2025-06-25	30	28654.31	35 days
2	CUST10002	2025-07-12	24	23884.03	18 days
3	CUST10003	2025-05-10	25	24206.03	81 days
4	CUST10004	2025-07-22	19	25565.30	8 days
...	...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19	402 days
996	CUST10996	2025-07-15	21	21809.11	15 days
997	CUST10997	2025-06-28	20	21120.48	32 days
998	CUST10998	2025-03-26	25	29494.56	126 days
999	CUST10999	2025-07-25	23	22028.01	5 days

1000 rows × 5 columns

STEP 5: Score RFM

```
df_rfm['R_Score'] = pd.qcut(df_rfm['RecencyInDays'],5,labels=[5,4,3,2,1])
df_rfm['F_Score'] = pd.qcut(df_rfm['Frequency'].rank(),5,labels=[1,2,3,4,5])
df_rfm['M_Score'] = pd.qcut(df_rfm['Monetary'],5,labels=[1,2,3,4,5])
df_rfm
```

	CustomerID	Recency	Frequency	Monetary	RecencyInDays	R_Score	F_Score	M_Score
0	CUST10000	2025-07-17	23	21265.49	13 days	4	3	2
1	CUST10001	2025-06-25	30	28654.31	35 days	3	5	5
2	CUST10002	2025-07-12	24	23884.03	18 days	4	3	3
3	CUST10003	2025-05-10	25	24206.03	81 days	1	4	3
4	CUST10004	2025-07-22	19	25565.30	8 days	5	1	4
...	...	...	...	...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19	402 days	1	2	3
996	CUST10996	2025-07-15	21	21809.11	15 days	4	2	3
997	CUST10997	2025-06-28	20	21120.48	32 days	3	2	2
998	CUST10998	2025-03-26	25	29494.56	126 days	1	4	5
999	CUST10999	2025-07-25	23	22028.01	5 days	5	3	3

1000 rows × 8 columns

STEP 6: Create Combined RFM Segment

```
df_rfm['Segment'] = df_rfm['R_Score'].astype('string')+df_rfm['F_Score'].astype('string')+df_rfm['M_Score'].astype('string')
df_rfm
```

	CustomerID	Recency	Frequency	Monetary	RecencyInDays	R_Score	F_Score	M_Score	Segment
0	CUST10000	2025-07-17	23	21265.49	13 days	4	3	2	432
1	CUST10001	2025-06-25	30	28654.31	35 days	3	5	5	355
2	CUST10002	2025-07-12	24	23884.03	18 days	4	3	3	433
3	CUST10003	2025-05-10	25	24206.03	81 days	1	4	3	143
4	CUST10004	2025-07-22	19	25565.30	8 days	5	1	4	514
...	...	...	...	...	...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19	402 days	1	2	3	123
996	CUST10996	2025-07-15	21	21809.11	15 days	4	2	3	423
997	CUST10997	2025-06-28	20	21120.48	32 days	3	2	2	322
998	CUST10998	2025-03-26	25	29494.56	126 days	1	4	5	145
999	CUST10999	2025-07-25	23	22028.01	5 days	5	3	3	533

1000 rows × 9 columns

```
df_rfm.dtypes
```

```
CustomerID      str
Recency         datetime64[us]
Frequency       int64
Monetary        float64
RecencyInDays   timedelta64[us]
R_Score         category
F_Score         category
M_Score         category
Segment         string
dtype: object
```

Step 7: Assign Segment Labels

```
conditions = [
    df_rfm['Segment'].str.match(r'^[45][45][45]'),#Champions
    df_rfm['Segment'].str.match(r'^[2345][45].'),#Loyal Customer
    df_rfm['Segment'].str.match(r'^[12][345].'),#At Risk
    df_rfm['Segment'].str.match(r'^[34][23][45]'),#Hibernating
    df_rfm['Segment'].str.match(r'^[45][23].'),#Potential loyalist
    df_rfm['Segment'].str.match(r'^[1][12][12]')#Lost
labels = [
    'Champions',
    'Loyal Customer',
    'At Risk',
    'Hibernating',
    'Potential loyalist',
    'Lost',]
df_rfm['SegmentLabel'] = np.select(conditions,labels,default = 'Visitor')
df_rfm
```

	CustomerID	Recency	Frequency	Monetary	RecencyInDays	R_Score	F_Score	M_Score	Segment	SegmentLabel
0	CUST10000	2025-07-17	23	21265.49	13 days	4	3	2	432	Potential loyalist
1	CUST10001	2025-06-25	30	28654.31	35 days	3	5	5	355	Loyal Customer
2	CUST10002	2025-07-12	24	23884.03	18 days	4	3	3	433	Potential loyalist
3	CUST10003	2025-05-10	25	24206.03	81 days	1	4	3	143	At Risk
4	CUST10004	2025-07-22	19	25565.30	8 days	5	1	4	514	Visitor
...	...	...	...	...	...	...	...	...	...	...
995	CUST10995	2024-06-23	21	24325.19	402 days	1	2	3	123	Visitor
996	CUST10996	2025-07-15	21	21809.11	15 days	4	2	3	423	Potential loyalist
997	CUST10997	2025-06-28	20	21120.48	32 days	3	2	2	322	Visitor
998	CUST10998	2025-03-26	25	29494.56	126 days	1	4	5	145	At Risk
999	CUST10999	2025-07-25	23	22028.01	5 days	5	3	3	533	Potential loyalist

1000 rows × 10 columns

```
print(df_rfm[df_rfm['SegmentLabel'] == 'Visitor']['Segment'].unique())
```

```
<StringArray>  
['514', '512', '411', '323', '212', '213', '511', '211', '125', '311', '223',  
 '312', '412', '222', '333', '221', '414', '214', '123', '124', '322', '513',  
 '331', '332', '224', '225', '113', '313', '321', '215', '115']  
Length: 31, dtype: string
```

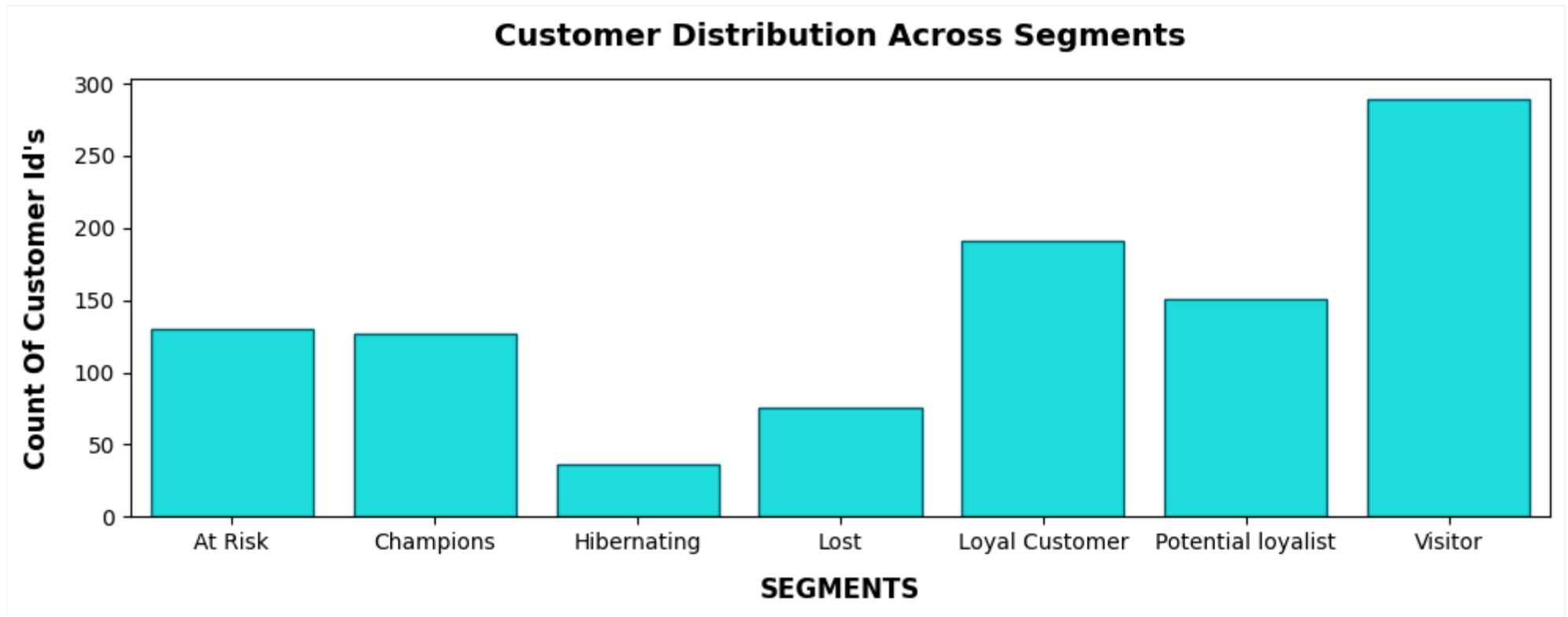
STEP 8: Visualise

Count of customers in each segment

```
countOfCustomerForEachSegment = df_rfm.groupby(['SegmentLabel'])['CustomerID'].count().reset_index()  
countOfCustomerForEachSegment
```

	SegmentLabel	CustomerID
0	At Risk	130
1	Champions	127
2	Hibernating	36
3	Lost	76
4	Loyal Customer	191
5	Potential loyalist	151
6	Visitor	289

```
plt.figure(figsize = (10,4))
sns.barplot(x = countOfCustomerForEachSegment['SegmentLabel'], y = countOfCustomerForEachSegment['CustomerID'],color = 'cyan',edgecolor = '#005b66')
plt.title('Customer Distribution Across Segments', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('SEGMENTS', fontsize=12, fontweight='bold', labelpad=10)
plt.ylabel("Count Of Customer Id's", fontsize=12, fontweight='bold', labelpad=10)
plt.tight_layout()
plt.show()
```

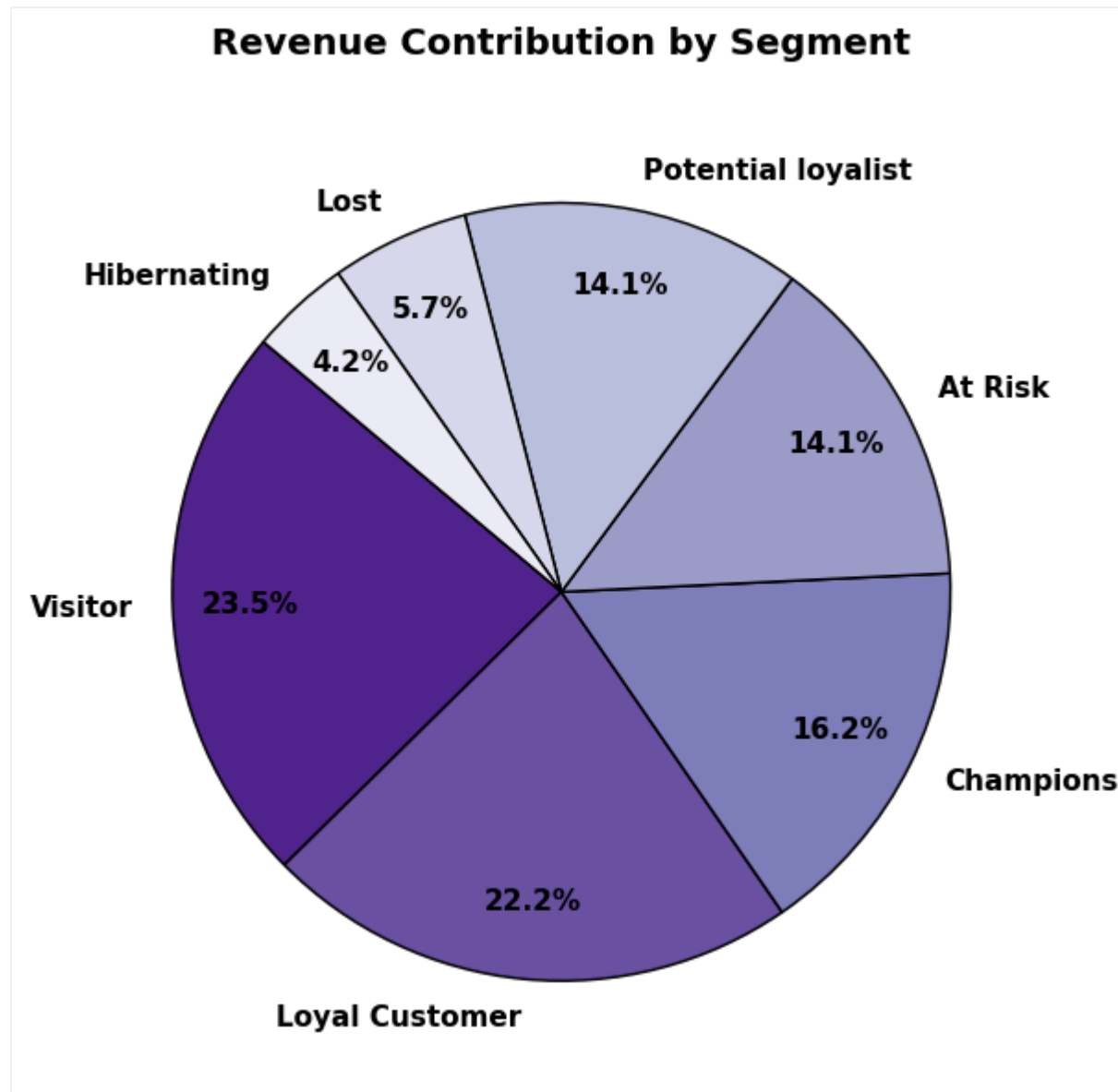


Revenue contribution per segment

```
RevenueContriPerSeg = df_rfm.groupby(['SegmentLabel'])[['Monetary']].sum()
RevenueContriPerSeg = RevenueContriPerSeg.sort_values(by='Monetary', ascending=False)
RevenueContriPerSeg
```

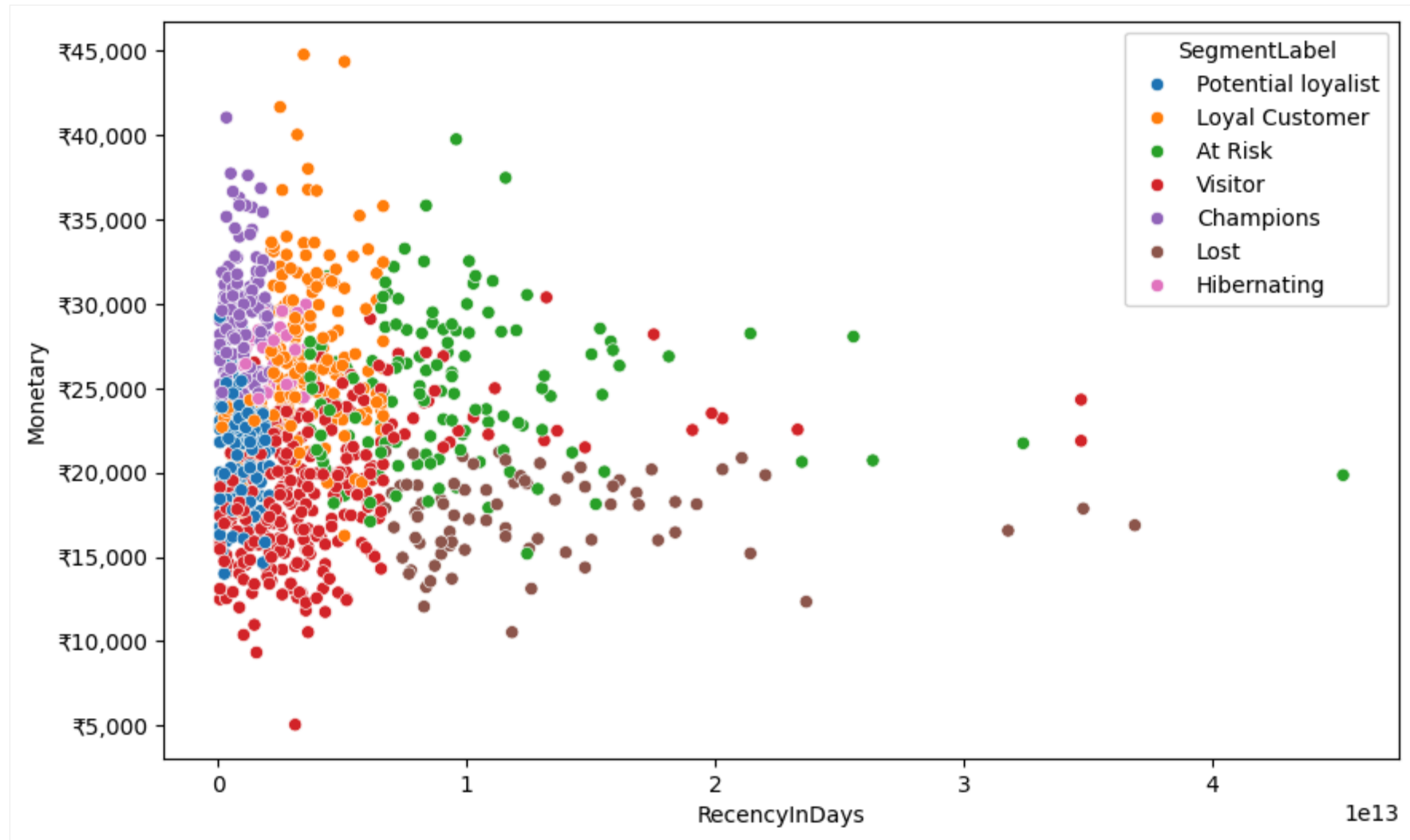
	Monetary
SegmentLabel	
Visitor	5416380.24
Loyal Customer	5122606.00
Champions	3725144.54
At Risk	3257085.87
Potential loyalist	3250819.39
Lost	1323163.04
Hibernating	958000.58

```
plt.figure(figsize=(10,6))
colors = sns.color_palette('Purples_r', len(RevenueContriPerSeg))
plt.pie(RevenueContriPerSeg['Monetary'], labels = RevenueContriPerSeg.index, autopct = '%1.1f%%', colors = colors, startangle= 140, pctdistance=0.80, textprops=
{'fontsize': 11, 'weight': 'bold'}, wedgeprops={'edgecolor': 'black', 'linewidth': 1})
plt.title('Revenue Contribution by Segment', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
```



Recency vs Monetary scatter plot colored by segment

```
plt.figure(figsize =(10,6))
ax = sns.scatterplot(x = df_rfm['RecencyInDays'], y = df_rfm['Monetary'],hue = df_rfm['SegmentLabel'])
ax.yaxis.set_major_formatter(ticker.StrMethodFormatter('₹{x:,.0f}'))
```

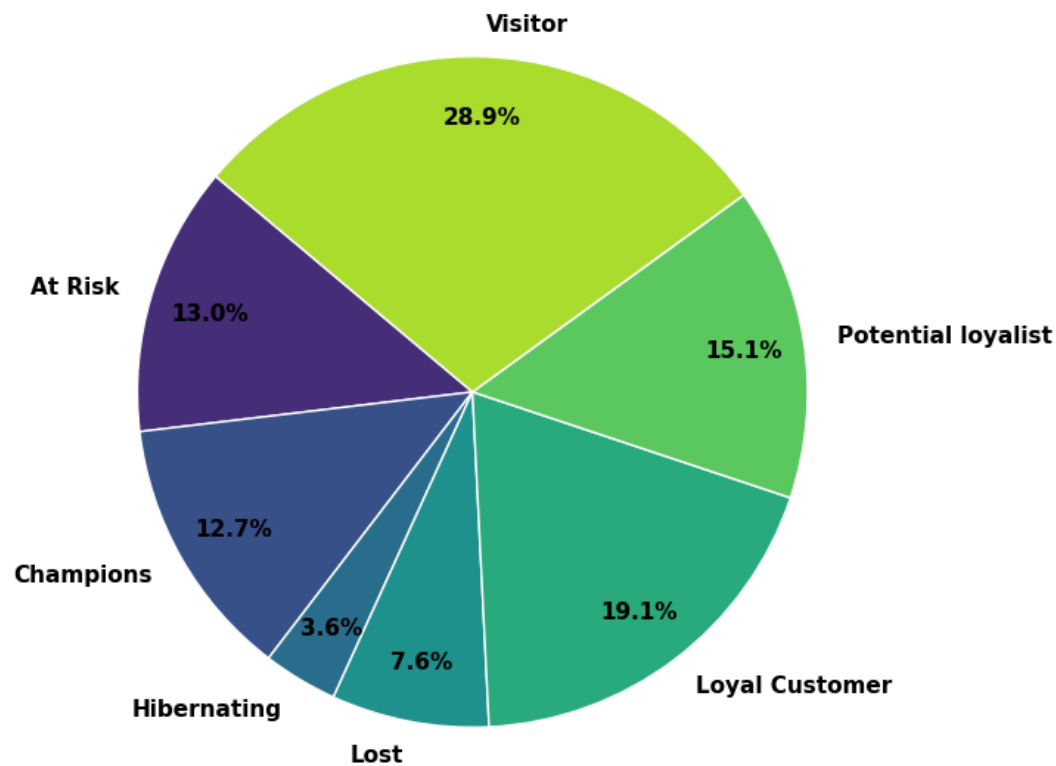


```

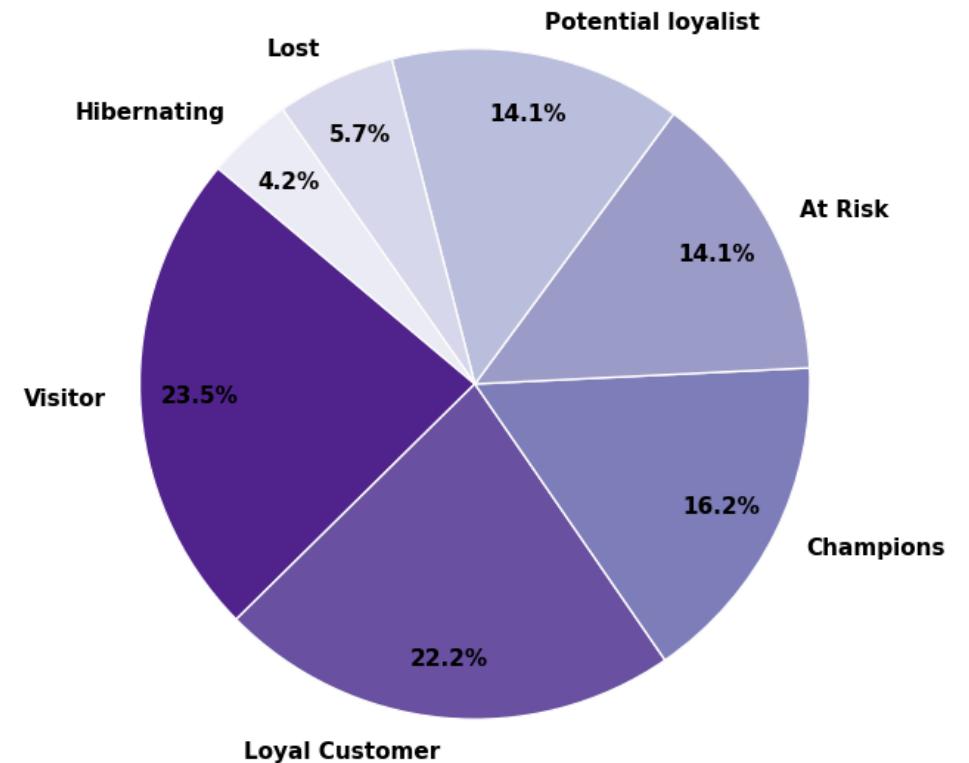
fig, ax = plt.subplots(1,2,figsize=(14,8))
colors1 = sns.color_palette('viridis', len(countOfCustomerForEachSegment))
colors2 = sns.color_palette('Purples_r', len(RevenueContriPerSeg))
ax[0].pie(countOfCustomerForEachSegment['CustomerID'], labels = countOfCustomerForEachSegment['SegmentLabel'],colors = colors1, autopct = '%1.1f%%',startangle=
140,pctdistance=0.82,textprops={'fontsize': 11, 'weight': 'bold'}, wedgeprops={'edgecolor': 'white', 'linewidth': 1})
ax[1].pie(RevenueContriPerSeg['Monetary'], labels = RevenueContriPerSeg.index, autopct = '%1.1f%%',colors = colors2,startangle= 140,pctdistance=0.82,textprops=
{'fontsize': 11, 'weight': 'bold'}, wedgeprops={'edgecolor': 'white', 'linewidth': 1})
ax[0].set_title('Percentage of Customers in each Segment', fontsize = 14, fontweight = 'bold', pad =20)
ax[1].set_title('Revenue Contribution by Segment', fontsize = 14, fontweight = 'bold', pad = 20)
plt.tight_layout()
plt.show()

```

Percentage of Customers in each Segment



Revenue Contribution by Segment



```

if 'SegmentLabel' in countOfCustomerForEachSegment.columns:
    count_df = countOfCustomerForEachSegment.set_index('SegmentLabel')
else:
    count_df = countOfCustomerForEachSegment.copy()

df_analysis = pd.DataFrame({
    'Revenue': RevenueContriPerSeg['Monetary'],
    'CountOfCustomer': count_df['CustomerID']
}).sort_values(by='Revenue', ascending=False)

df_analysis['Cum_Revenue'] = df_analysis['Revenue'].cumsum()
df_analysis['Cum_Customers'] = df_analysis['CountOfCustomer'].cumsum()

df_analysis['Cum_Revenue_%'] = (df_analysis['Cum_Revenue'] / df_analysis['Revenue'].sum()) * 100
df_analysis['Cum_Customers_%'] = (df_analysis['Cum_Customers'] / df_analysis['CountOfCustomer'].sum()) * 100

styled_df = df_analysis.style.format({
    'Revenue': '₹{:, .2f}',
    'CountOfCustomer': '{:, .0f}',
    'Cum_Revenue': '₹{:, .2f}',
    'Cum_Customers': '{:, .0f}',
    'Cum_Revenue_%': '{:.1f}%',
    'Cum_Customers_%': '{:.1f}%'
})

styled_df

```

	Revenue	CountOfCustomer	Cum_Revenue	Cum_Customers	Cum_Revenue_%	Cum_Customers_%
SegmentLabel						
Visitor	₹5,416,380.24	289	₹5,416,380.24	289	23.5%	28.9%
Loyal Customer	₹5,122,606.00	191	₹10,538,986.24	480	45.7%	48.0%
Champions	₹3,725,144.54	127	₹14,264,130.78	607	61.9%	60.7%
At Risk	₹3,257,085.87	130	₹17,521,216.65	737	76.0%	73.7%
Potential loyalist	₹3,250,819.39	151	₹20,772,036.04	888	90.1%	88.8%
Lost	₹1,323,163.04	76	₹22,095,199.08	964	95.8%	96.4%
Hibernating	₹958,000.58	36	₹23,053,199.66	1,000	100.0%	100.0%

ANALYSIS

1. Since highest revenue comes from Visitor section where either Recency, Frequency or Monetary has the lowest score, We can make offers available for first time consumers to increase their RFM score.
2. To retain Champions and Loyalists exclusive offers can be provided to retain their trust
3. Visitors having higher monetary scores but lower recency or frequency. Thus they can be sent messages informing them of various offers
4. Hibernating customers can be sent messages to revive their account
5. Loyal customers and Champions follow Visitors in revenue. Thus it becomes important to inform them of any upcoming sales beforehand.
6. Scatterplot displays that substantial amount of transaction happens where Recency has the lowest score. Thus most orders under every segment get placed after a gap of few days
7. Loyal customers and Champions spend the highest among all the others
8. Visitors spend the lowest in aggregate, among all the groups
9. Loyal customers' expenses are far more equally distributed when it comes to expenses
10. 73% of the customer generate 76% of the revenue. Hence revenue contribution is uniform leaving space for different segments to upgrade to Loyalist or Champions

